SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

삼성청년
SW·AI아카데미

backend - Cookie & Session

SAMSUNG SW AI ACADEMY FOR YOUTH

목차

1. Cookie
2. Session
3. Exception 처리

부록: Session 활용

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Cookie

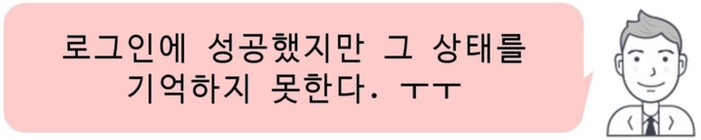
HTTP 특징

Confidential

❖ HTTP 특징

- Stateless: 상태를 기억하지 않음





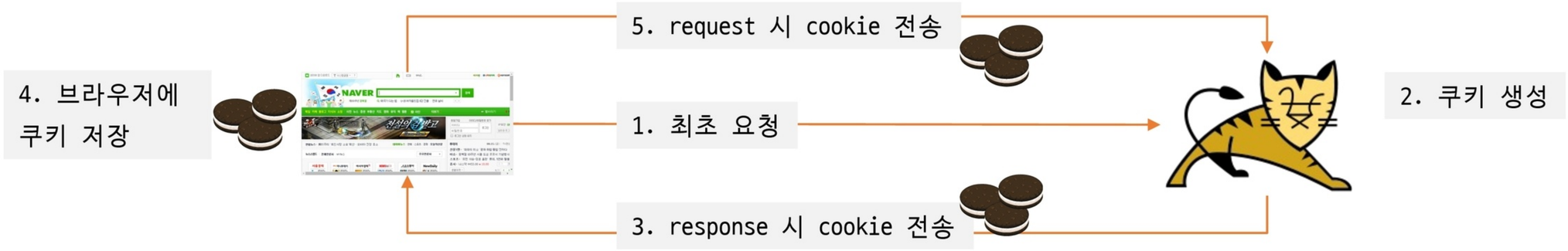
- Stateless 한 이유
  - ◆ 단순성: 각 요청이 독립적으로 처리되기 때문에 이전 요청과 연결할 필요가 없고 서버의 구성이 단순해지며 속도가 빠름
  - ◆ 확장성: 여러 서버 간 공유해야 할 상태가 없기 때문에 여러 서버에 요청을 분산시킬 수 있어 부하 분산 및 시스템 확장이 용이
  - ◆ 신뢰성: 요청들이 독립적이기 때문에 한 요청의 실패가 다른 요청에 영향을 미치지 않으므로 시스템 신뢰도 향상
  - ◆ 자원절약: 서버에서 상태를 저장하지 않기 때문에 그만큼 서버의 메모리, 저장 공간 절약

- Cookie, Session 등으로 stateless 특성 보완

Confidential

❖ Cookie

- 웹 서버에서 정보를 생성해서 클라이언트(웹 브라우저)에 보관하는 데이터



- 쿠키의 구현
  - ◆ jakarta.servlet.http.Cookie

Confidential

❖ Cookie의 주요 property

- name은 getter만 제공, 나머지 property는 getter/setter로 접근 가능

- name: 쿠키를 만들 때 전달하는 쿠키의 이름으로 각각의 쿠키를 구별하는 유일한 값
  - ◆ 동일한 이름의 쿠키는 기존 쿠키를 덮어씀
  - ◆ 알파벳, 숫자, 하이픈, 언더스코어, 물결, 점으로 구성되며 공백 등 나머지는 URLEncoder를 통해 인코딩 필요

- value: 쿠키의 값으로 쿠키 생성시 전달하거나 setValue() 메서드를 통해 설정 가능
  - ◆ 작성 규칙은 name과 동일하며 추가로 한글 사용 가능

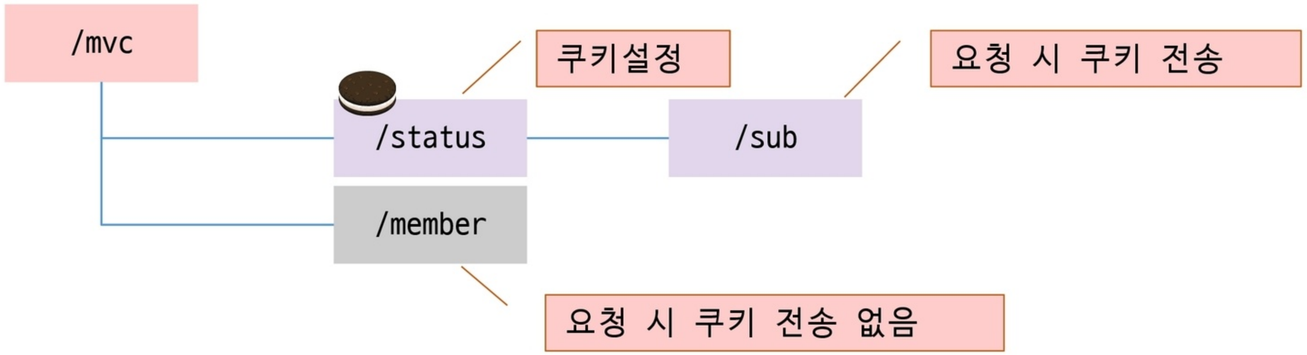
```
Cookie cookie1 = new Cookie("username", "hong_gil_dong");

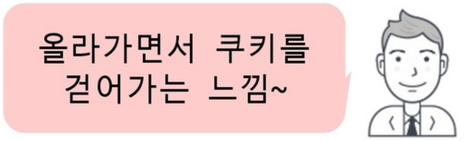
Cookie cookie2 = new Cookie("status-path", URLEncoder.encode("status 경로에 저장된 쿠키", "UTF-8"));
```

Confidential

❖ Cookie의 주요 property

- path: 쿠키가 유효한 경로
  - ◆ path가 설정된 하위 경로 요청에 대해서만 쿠키 전송





  - ◆ 경로 미설정 시: context root 설정
  - ◆ / 즉 container root로 지정하면 동일 도메인의 다른 애플리케이션까지 접근
    - localhost:8080/out_app, localhost:8080/your_app은 도메인이 같으므로 쿠키 공유 - 보안 이슈 발생 가능

```
        domain
      ──────────
http://localhost:8080
```

Confidential

❖ Cookie의 주요 property

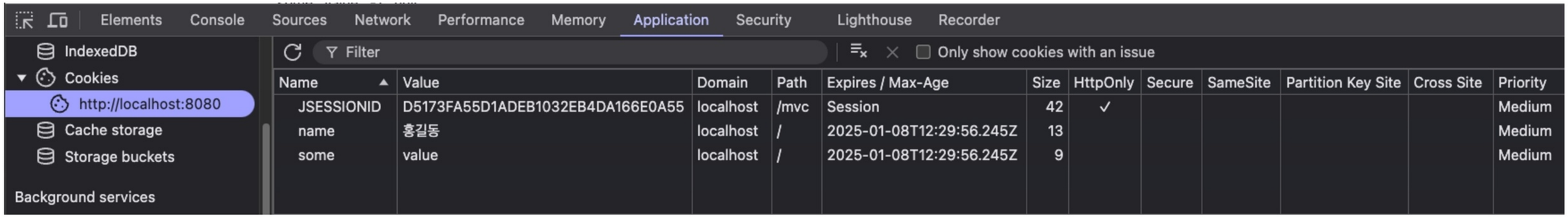
- maxAge: 쿠키의 유효 기간
  - ◆ 양수: 초 단위로 해당 시간까지 쿠키 존재, 시간이 지나면 자동 폐기
  - ◆ 음수 또는 미지정 시: 세션 쿠키로 브라우저 종료 등으로 세션 종료 시 폐기
  - ◆ 0: 브라우저에 도착하는 즉시 폐기
- 보안 옵션 3종 세트
  - ◆ secure: HTTPS에서만 전송 허용 → 쿠키 가로채기 방지
  - ◆ httpOnly: 서버만 접근 가능하며 JavaScript에서 접근 불가 설정 → XSS 공격 방지
  - ◆ SameSite: 쿠키 전송 가능 사이트 제한(Strict, Lax, None) → CSRF 공격 방지
- 개발자 도구에서 쿠키 확인: Application > Cookies

쿠키는 특별히 삭제 메서드가 없고
유효기간이 0인 쿠키로 덮어쓴다.



Chrome의 경우 최대 유효 기간을
400일로 제한한다.





Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Partition Key Site	Cross Site	Priority
JSESSIONID	D5173FA55D1ADEB1032EB4DA166E0A55	localhost	/mvc	Session	42	✓					Medium
name	홍길동	localhost	/	2025-01-08T12:29:56.245Z	13						Medium
some	value	localhost	/	2025-01-08T12:29:56.245Z	9						Medium

Confidential

❖ Cookie의 생성과 사용

```
// make-cookie
Cookie cookie = new Cookie("user", "andy");
cookie.setMaxAge(60 * 1);
response.addCookie(cookie);

Cookie cookie2 = new Cookie("addr", URLEncoder.encode("서울특별시 강남구", "utf-8"));
cookie2.setMaxAge(-1);
response.addCookie(cookie2);

Cookie cookie3 = new Cookie("phone", "010-0000-0000");
cookie3.setMaxAge(0);
response.addCookie(cookie3);
```

Cookie의 생성도 중요하지만
클라이언트에 전달되어야 한다.



쿠키는 요청을 통해 서버로
전송된다. forward된 page에서
즉시 확인 가능할까?



```
// check-cookie.jsp
Cookie [] cookies = request.getCookies();
if(cookies!=null) {
  for(Cookie c: cookies) {// Client -> Server로는 name과 value 만 전송
    System.out.println(c.getName()+" : "+ URLDecoder.decode(c.getValue(), "utf-8"));
  }
}
```

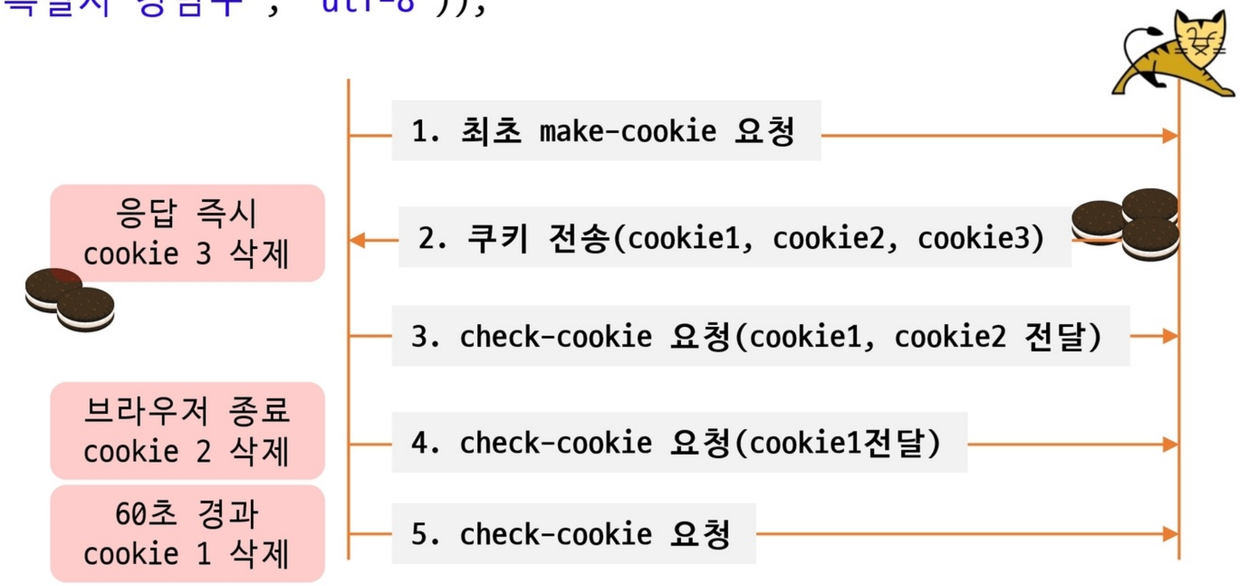
Confidential

❖ Cookie의 생성(TODO:01~03)

```
// make-cookie
Cookie cookie = new Cookie("user", "andy");
cookie.setMaxAge(60 * 1);
response.addCookie(cookie);

Cookie cookie2 = new Cookie("addr", URLEncoder.encode("서울특별시 강남구", "utf-8"));
cookie2.setMaxAge(-1);
response.addCookie(cookie2);

Cookie cookie3 = new Cookie("phone", "010-0000-0000");
cookie3.setMaxAge(0);
response.addCookie(cookie3);
```



Confidential

❖ Cookie의 활용(TODO:04)

- Servlet: request의 getCookies() 활용

```
Cookie [] cookies = request.getCookies();
if(cookies!=null) {
  for(Cookie c: cookies) {// Client -> Server로는 name과 value 만 전송
    System.out.println(c.getName()+" : "+ URLDecoder.decode(c.getValue(), "utf-8"));
  }
}
```

- JSP: EL 내장 객체 cookie 활용

```
<h1>개별 쿠키 확인(EL의 쿠키 내장 객체 활용): </h1>
<span>for-domain: ${cookie["just-10-min"].value }</span>
```

```
<c:if test="${cookie != null}">
  <c:forEach items="${cookie}" var="entry">
    <tr>
      <td>${entry.value.name }</td>
      <td>${URLDecoder.decode(entry.value.value, "utf-8") }</td>
    </tr>
  </c:forEach>
</c:if>
```

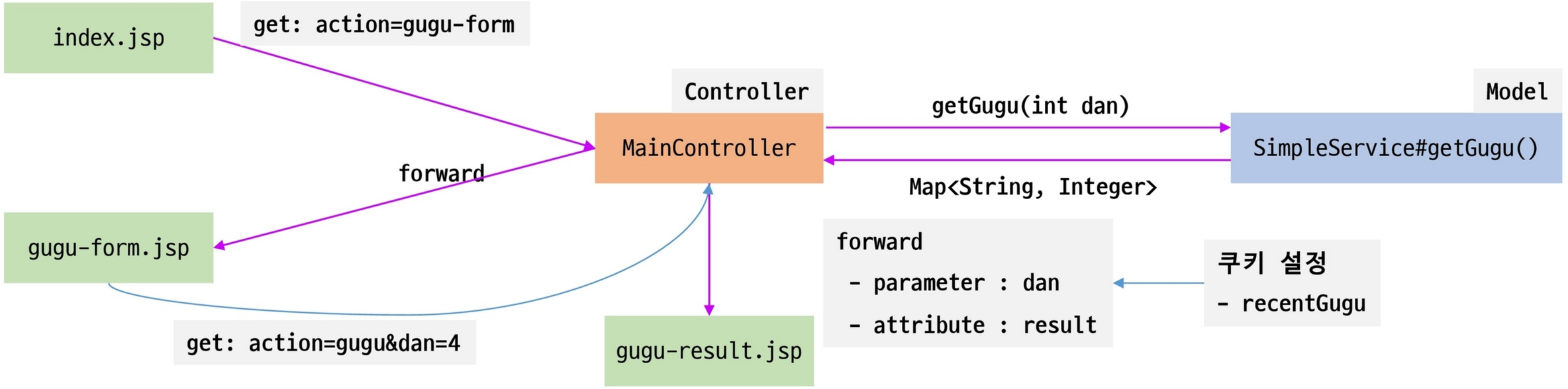
이건 Map

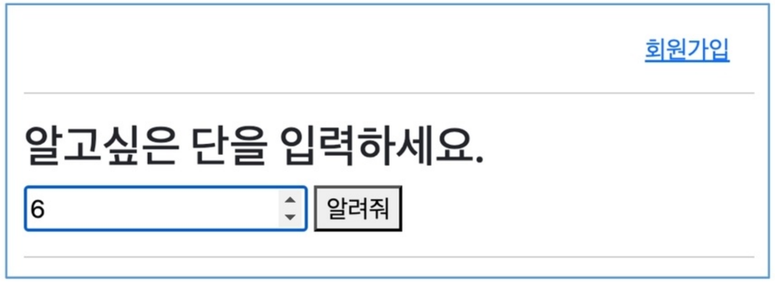
이게 Cookie

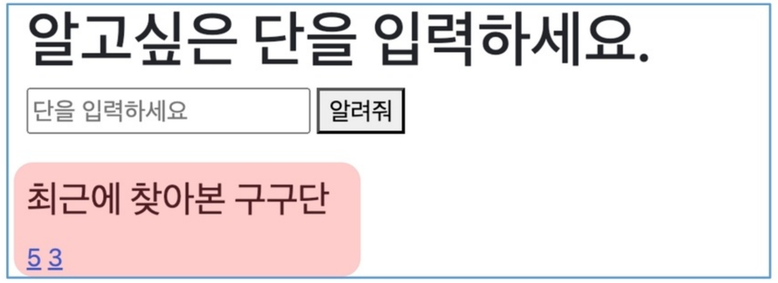
Confidential

❖ Gugu History(TODO:05,06)

- 최근 호출한 구구단 기억해 두기







Confidential

❖ Cookie의 주요 property

- SameSite(2016년 제안): 쿠키 전송 가능 사이트 제한(Strict, Lax, None) → CSRF 공격 방지
  - ◆ Strict: 같은 사이트 요청에서만 쿠키 전송 → CSRF 완벽 방지 But! 사용성이 떨어짐
  - ◆ Lax: Strict + 일부 cross site 요청에서 쿠키 전송(기본 값) → CSRF 를 어느정도 방지하면서 사용성도 어느정도 유지
  - ◆ None: 제약 없음 → CSRF 위험이 증가하므로 Secure=true 인 상태에서만 가능
    - 광고 추적, 결제 게이트웨이 등 서드파티 쿠키가 필요할 때만

요청 상황 (출발지 → 목적지)	요청 방식	Strict	Lax (기본값)	None (+Secure)
주소창 직접 입력 / 북마크	Direct	✅ 전송	✅ 전송	✅ 전송
같은 사이트 내 이동 (co → co)	모든 방식	✅ 전송	✅ 전송	✅ 전송
타 사이트에서 링크 클릭 (google → co)	 cookie owner site	❌ **차단**	✅ **전송**	✅ 전송
타 사이트에서 폼 전송 (GET)	<form method="GET" action="co">	❌ **차단**	✅ **전송**	✅ 전송
타 사이트에서 폼 전송 (POST)	<form method="POST" action="co">	❌ 차단	❌ **차단**	✅ 전송
타 사이트에서 이미지 로딩	<img>, <iframe>	❌ 차단	❌ 차단	✅ 전송
타 사이트에서 비동기 요청	AJAX / fetch	❌ 차단	❌ 차단	✅ 전송

* co: cookie owner

Confidential

❖ setAttribute()를 통한 쿠키 설정(Servlet 6.0+)

- 기본 규칙

```
Cookie cookie = new Cookie("sessionId", "abc123");

// 1. 값이 있는 속성: "속성=값" 형태
cookie.setAttribute("SameSite", "Strict");    // SameSite=Strict;
cookie.setAttribute("Domain", ".mysite.com"); // Domain=.mysite.com;

// 2. 값이 없는 속성 (boolean 속성) - "속성"만 표시
cookie.setAttribute("HttpOnly", "");    // HttpOnly;
cookie.setAttribute("Secure", "");      // Secure;

// 3. 속성 제거 (아예 없어짐)
cookie.setAttribute("SameSite", null);  // SameSite 사라짐

**최종 Set-Cookie 헤더:**
Set-Cookie: sessionId=abc123; HttpOnly; Secure;
```

- 장점
  - ◆ 일관되게 모든 쿠키 속성 설정 가능
  - ◆ SameSite 등 나중에 추가된 속성도 손쉽게 설정 가능

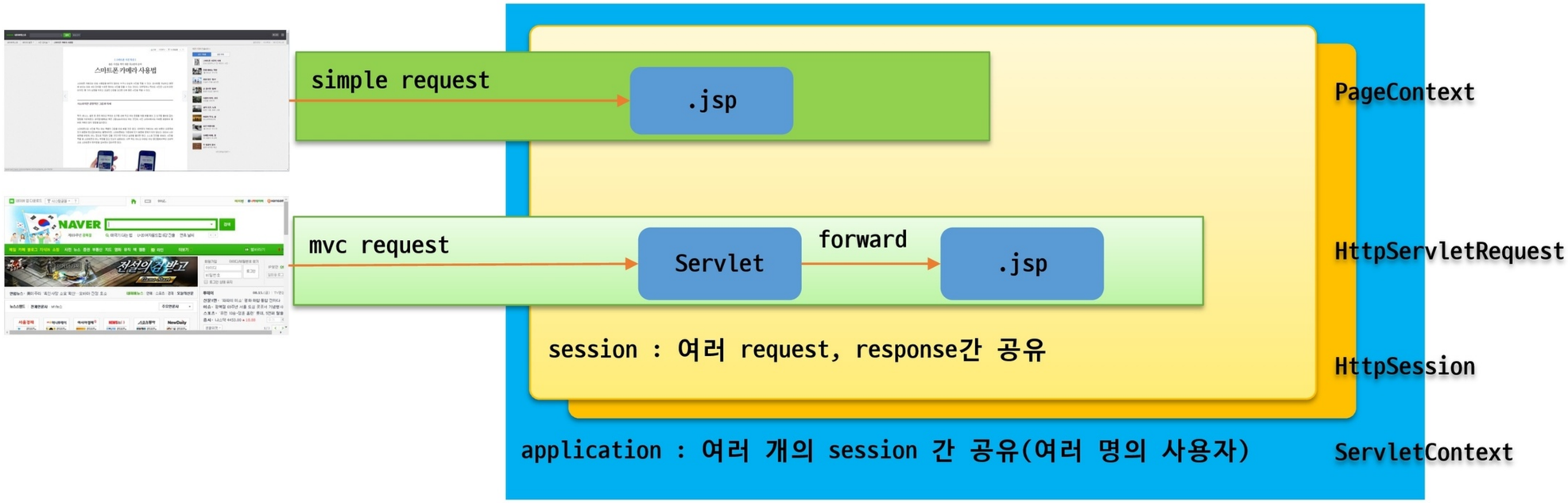
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Session

Confidential

❖ 웹의 기본 Scope



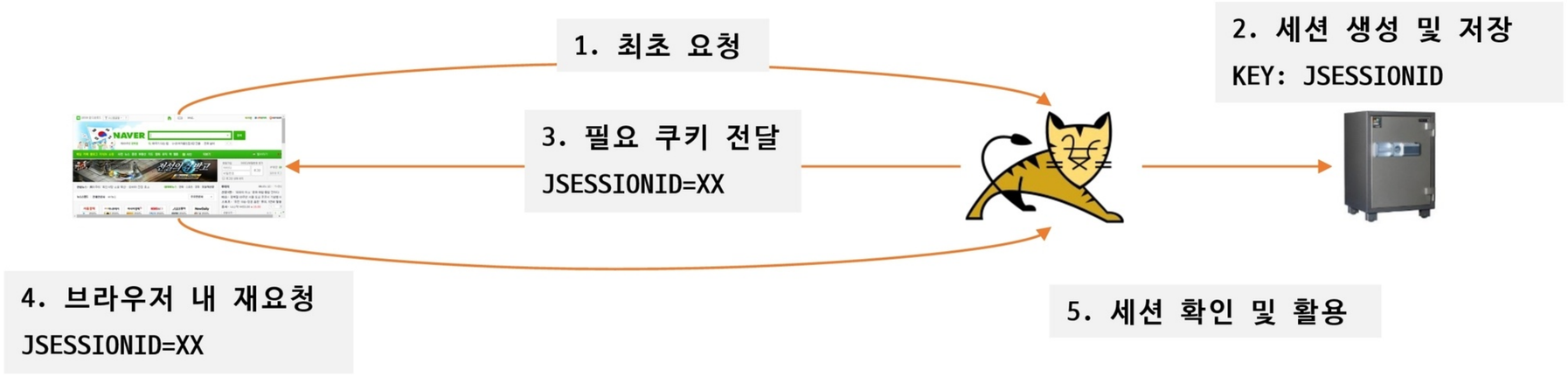
page 영역	하나의 page → 페이지 내에서만 존재(EL의 기본 scope)
request 영역	하나의 request → 여러 page로 구성 → 하나의 요청이 끝날 때(응답할 때)까지 존재
session 영역	하나의 session → 여러 request로 구성 → 브라우저를 닫는 등 세션이 종료될 때까지 존재
application 영역	하나의 application → 여러 session으로 구성 → 애플리케이션이 종료될 때까지 존재

Confidential

❖ Session

- 서버에 클라이언트의 상태 값을 저장
- 하나의 브라우저 당 하나의 세션 성립
- 쿠키는 브라우저를 닫아도 유지될 수 있으나 세션은 브라우저를 닫으면 종료

- 세션이 동작하기 위해서는 JSESSIONID라는 이름의 쿠키 필요
  - ◆ JSESSIONID는 서버의 세션 공간에 들어가기 위한 키!



Confidential

❖ HttpSession 사용

- Servlet에서 Session 객체 획득
  - ◆ request.getSession(): 현재의 세션을 반환하며 아직 없을 경우 새로 생성

- 주요 메서드

메서드 시그니처	설명
Object getAttribute(String name)	지정된 이름(name)으로 저장된 세션 객체 가져오기
void setAttribute(String name, Object value)	세션 속성 설정
void removeAttribute(String name)	name에 해당하는 세션 속성 제거
void invalidate()	세션 무효화, logout 처리
String getId()	세션 ID 가져오기

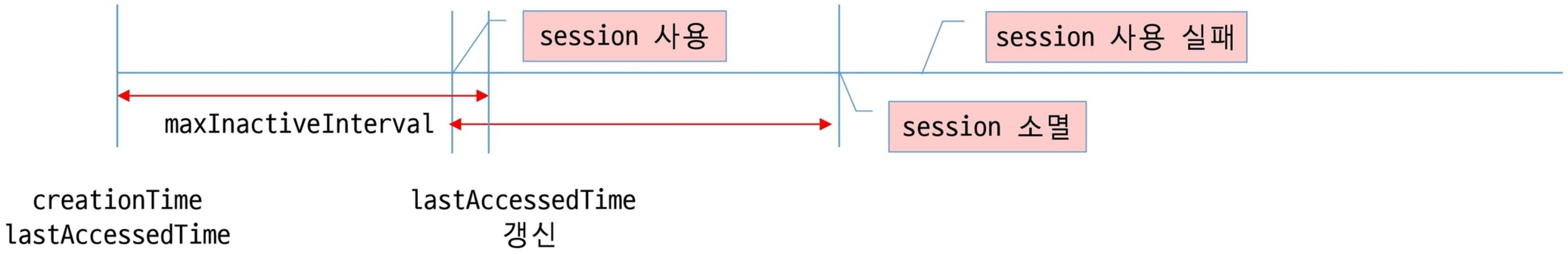
Confidential

❖ HttpSession의 유효기간

● 주요 메서드

메서드 시그니처	설명
`long getCreationTime()`	세션 생성 시각
`long getLastAccessedTime()`	세션 마지막 접근 시각
`int getMaxInactiveInterval()`	최대 세션 유효 기간
`void setMaxInactiveInterval(int interval)`	최대 세션 유효 기간 설정(초 단위)

◆ 세션은 getLastAccessedTime() 부터 getMaxInactiveInterval() 까지 유효

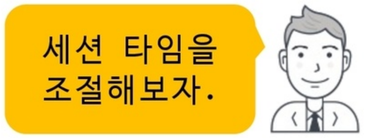


Confidential

❖ HttpSession의 유효기간 설정(TODO:07)

- WAS의 web.xml에 container 차원의 session-timeout 설정
  - ◆ 설정 시간은 분 단위

```
<session-config>
  <session-timeout>30</session-timeout>
</session-config>
```



- 애플리케이션 별 web.xml에서 context 단위 재정의 가능

- HttpSession#setMaxInactiveInterval(int sec)을 통해 프로그래밍적으로 재정의 가능
  - ◆ 초 단위로 세션의 유효 기간 설정

Confidential

❖ login/logout 처리



Confidential

❖ login 처리 - 화면(TODO: 08, 09)

- 상황에 따른 화면 처리:

```
<c:if test="${empty loginUser }">
  <a href="${root }/member?action=login-form" class="mx-3">로그인</a> |
  <a href="${root }/member?action=regist-member-form" class="mx-3">회원가입</a>
</c:if>

<c:if test="${!empty loginUser }">
  <a href="#" class="mx-3">${loginUser.name }</a> |
  <a href="${root }/member?action=logout" class="mx-3">로그아웃</a>
</c:if>
```

Confidential

❖ login 처리 - Controller(TODO: 10)

```
private void login(HttpServletRequest request, HttpServletResponse response)
                                                        throws ServletException, IOException {
  String email = request.getParameter("email");
  String pass = request.getParameter("pass");
  Member member = mService.login(email, pass);

  if (member != null) {
    request.getSession().setAttribute("loginUser", member);
    redirect(request, response, "/");
  } else {
    request.setAttribute("alertMsg", "로그인에 실패했습니다. 아이디와 비밀번호를 확인해주세요.");
    forward(request, response, "/member/login-form.jsp");
  }
}
```

Session 활용

Confidential

❖ logout 처리(TODO: 11)

```
private void logout(HttpServletRequest request, HttpServletResponse response)
                                                        throws ServletException, IOException {

  request.getSession().invalidate();
  redirect(request, response, "/");
}
```

session.invalidate()와
session.removeAttribute("loginUser")
의 차이는?

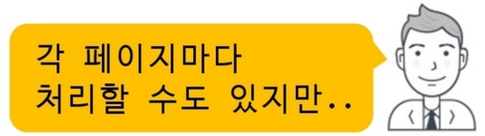


Session 활용

Confidential

❖ SessionFilter(TODO:12)

- 세션이 있는 상황에서만 접근해야 하는 페이지
  - ◆ ${root}/auth 하위 경로
- 경로 기반으로 filter를 이용해서 관리 가능




```
@WebFilter("/auth/*")
public class SessionFilter extends HttpFilter{
  @Override
  protected void doFilter(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
                                                                    throws IOException, ServletException {
    HttpSession session = req.getSession();
    if(session.getAttribute("loginUser")!=null) {
      chain.doFilter(req, res);
    }else {
      session.setAttribute("alertMsg", "로그인 후 사용하세요.");
      res.sendRedirect(req.getContextPath() + "/member?action=login-form");
    }
  }
}
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

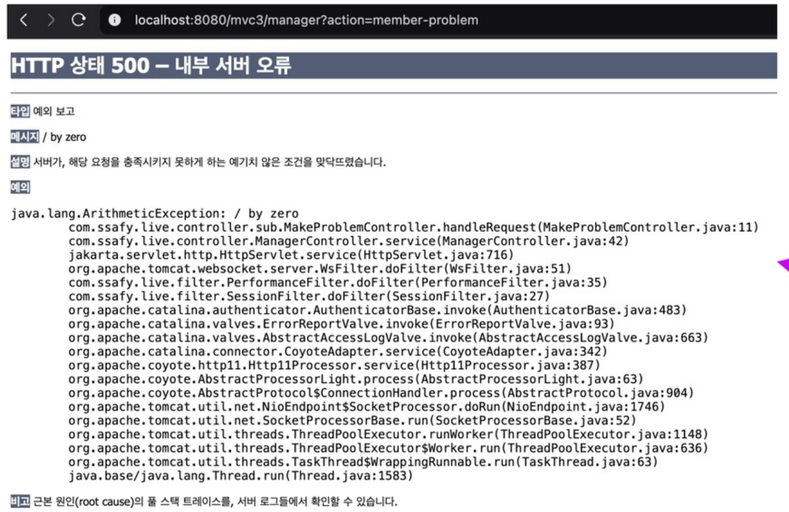
Exception 처리

Exception 처리

Confidential

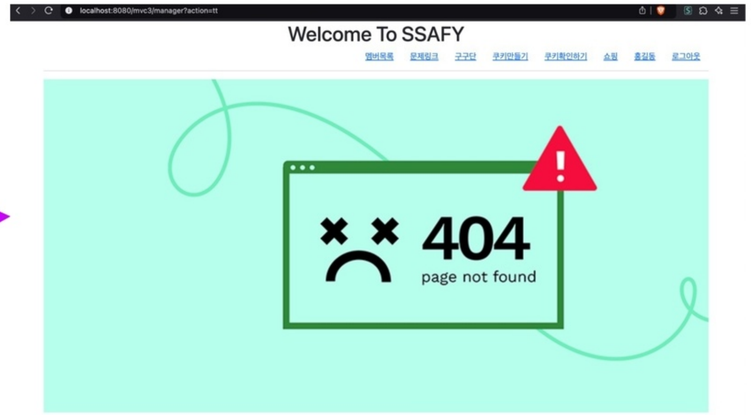
❖ 기본 Exception 처리 정책

- 어떠한 경우에도 예외에 대한 정보가 클라이언트에게 직접 전달되지 않도록 처리 필요
- [사용자에게 표시되는 페이지]와 [문제 해결을 위한 로깅]이 병행되어야 함



예외에 대한 정보가 클라이언트에 직접 전달





사용자 친화적 화면 구성

```
심각: ehandler.jsp
java.lang.ArithmeticException: / by zero
        at org.apache.jsp.problem_jsp._jspService(problem_jsp.java:70)
        at org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:70)
        at javax.servlet.http.HttpServlet.service(HttpServlet.java:725)
        at org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrap
```

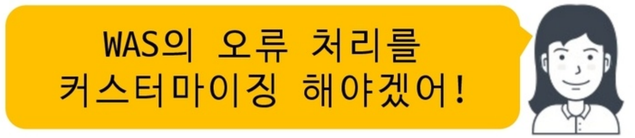
문제 해결을 위한 전문 로그

Exception 처리

Confidential

❖ 404 오류 처리 과정

- case 1: front controller까지 진입 후 처리할 sub controller를 발견하지 못한 경우
  - ◆ 존재하지 않는 경로의 예: http://localhost:8080/[context_name]/auth?action=not-exist-action
  - ◆ response에 404 에러 전달 → WAS가 기본 404 페이지로 처리
  - ◆ forward 처리 - 404 페이지로 이동 처리 가능




```
@Override
protected void service(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
  String action = getActionParameter(request, response);
  switch (action) {
    case "member-detail" -> memberDetail(request, response);
    //default -> response.sendError(HttpServletResponse.SC_NOT_FOUND); // WAS에 404 전달
    default -> forward(request, response, "/error/404.jsp");           // 지정된 페이지로 forward
  }
}
```

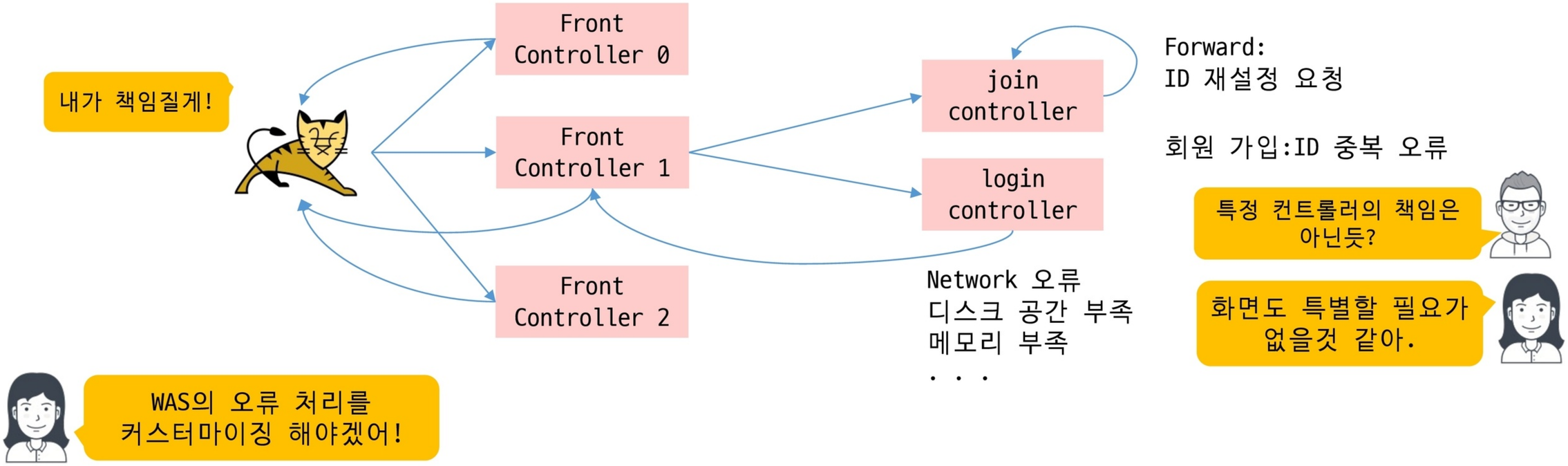
- case 2: front controller까지 진입하지 못한 요청
  - ◆ 존재하지 않는 경로의 예 : http://localhost:8080/[context_name]/not-exist-path
  - ◆ Servlet에 도달하지 않았으므로 처리 불가 → WAS가 기본 404 페이지로 처리

Exception 처리

Confidential

❖ 500 오류 처리 과정

- controller에서 try~catch 로 Exception 처리 후 직접 관련 페이지로 이동
- 시스템에서 예상치 못한 Unchecked Exception이 발생한다면? - fallback 필요



Exception 처리

Confidential

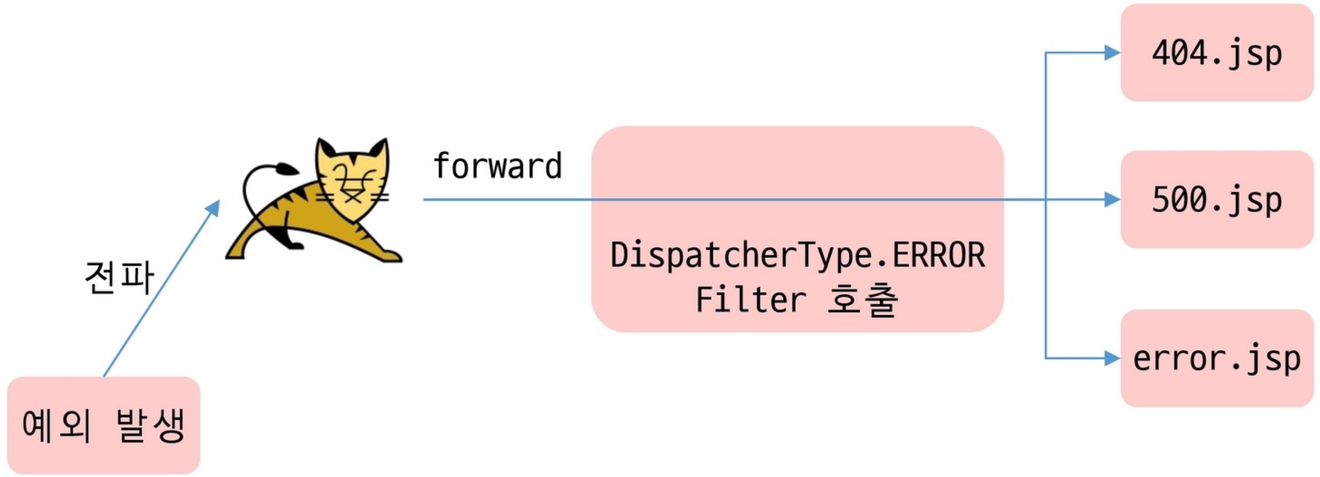
❖ WAS의 예외 처리 활용하기(TODO: 13~15)

- web.xml에 WAS가 받은 오류를 처리할 에러 페이지 설정

```
<error-page>
  <location>/error/error.jsp</location>
</error-page>

<error-page>
  <error-code>404</error-code>
  <location>/error/404.jsp</location>
</error-page>

<error-page>
  <error-code>500</error-code>
  <location>/error/500.jsp</location>
</error-page>
```



- WAS로 전달된 예외에 대한 모니터링
  - ◆ error page 호출 전 dispatcherTypes = {DispatcherType.ERROR} 가 설정된 Filter 호출

내일 수업을 위해서..

Confidential

❖ 실습 파일 설치

- 수업 git에서 BE_06_lab.zip 가져오기
- 사용할 workspace에 압축 풀기(여기에 압축 풀기)



- STS : [File] - [Open Projects from File System]
  - ◆ index.jsp 동작 확인

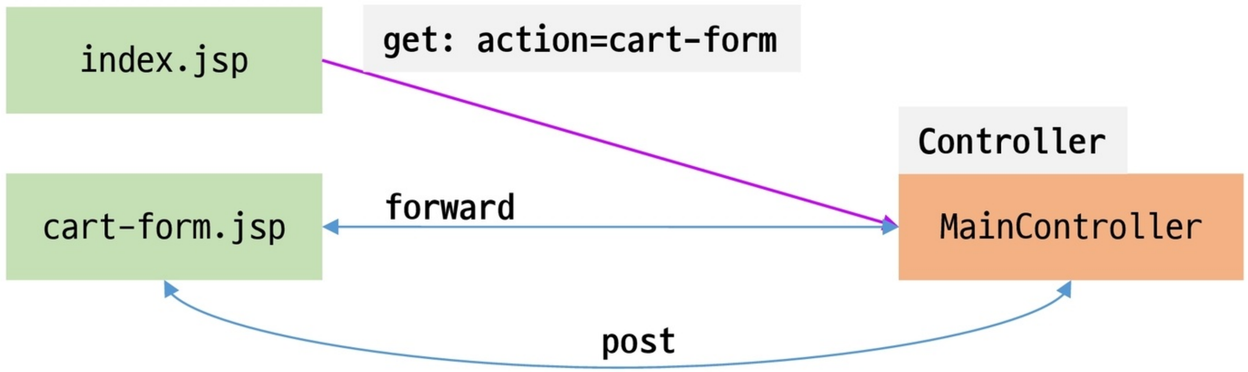
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

부록: session 활용

Confidential

❖ Shopping Cart 구현(TODO:16~17)

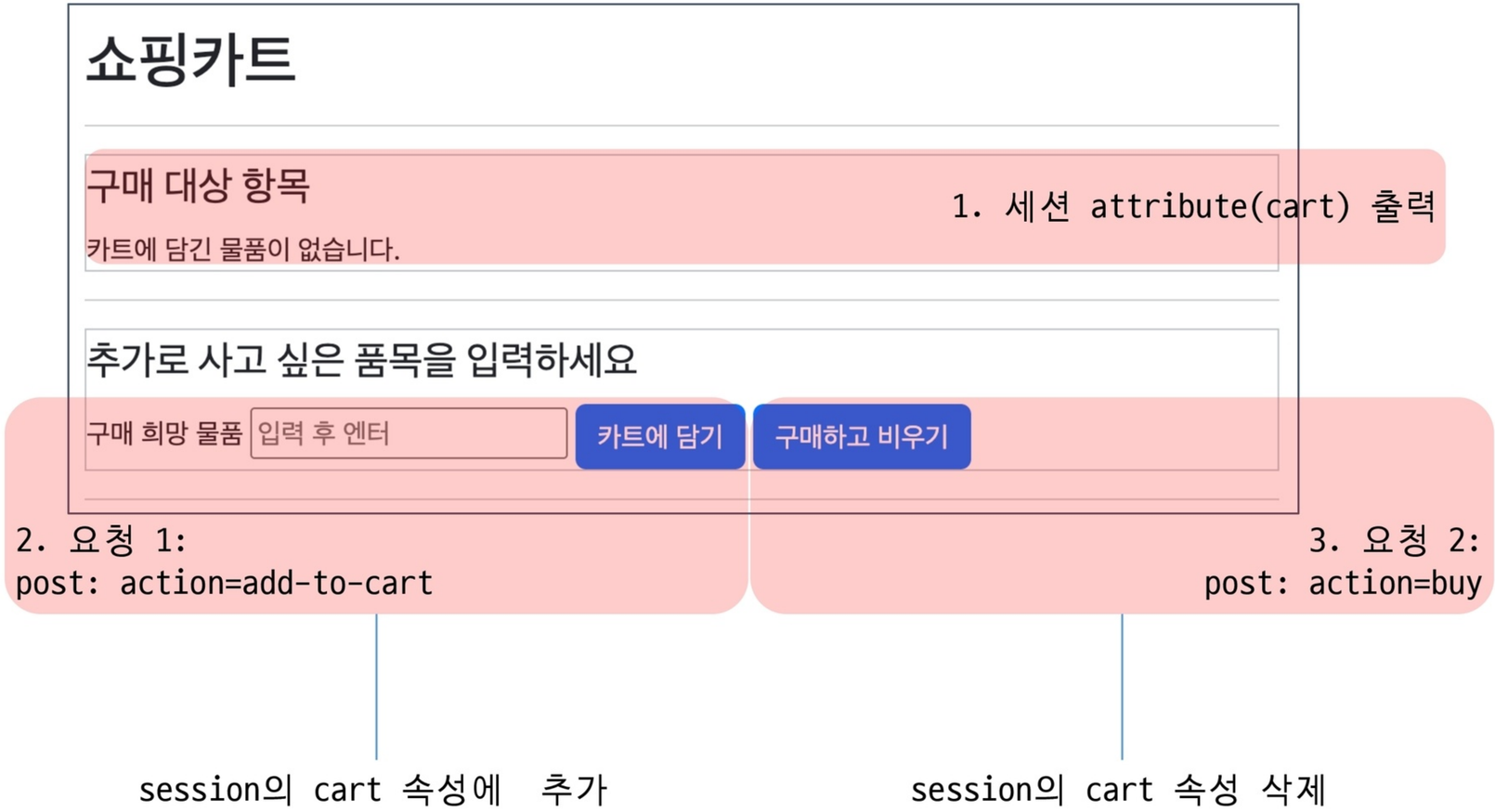


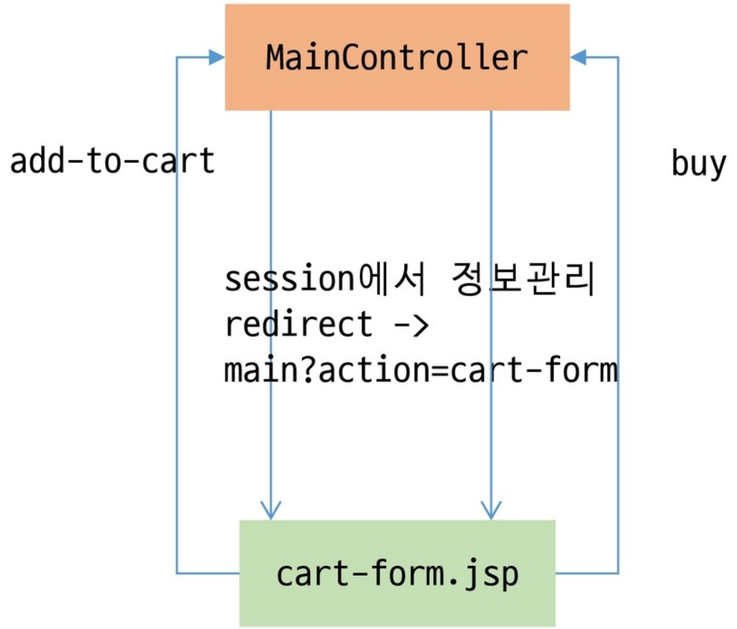


1. 세션 attribute(cart) 출력

Confidential

❖ Shopping Cart 구현(TODO:18~20)





SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

내일 방송에서 만나요!

삼성청년SW·AI아카데미